

claude-windows / 9f6357d8-0e35-491a-a830-052e094c7f37

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\9f6357d8-0e35-491a-a830-052e094c7f37\subagents\workflows\wf_635348d6-51c\agent-ae0be81534e590b52.jsonl`  
- SHA-256: `c34458c67e2bab40cbc39ff9bfebd4b615f8a520d59cc972d8ecef82868f069`  
- Source modified: `2026-06-21T13:56:02+00:00`  
- Imported at: `2026-07-05T16:48:25+00:00`  
- project: `wf_635348d6-51c`  
- session_id: `9f6357d8-0e35-491a-a830-052e094c7f37`
```

Transcript

1. user (2026-06-21T13:54:04.272Z)

In the engine repo at C:/Users/avidu/Projects/badminton-highlight-indexer, read the containerization/deploy for Cloud Run: deploy/cloudrun/Dockerfile and anything else under deploy/cloudrun/, plus deploy/gcp/ for any Cloud Run (gcloud run deploy/jobs) bits. Describe the Dockerfile (base image, python, deps, GPU/CUDA handling, whether/how it packages the native WASB torch-1.11 conda env, the entrypoint ? worker vs server). Is the image actually buildable/used today (build/push/deploy scripts? any CI that builds it?) or a scaffold? How does it handle Cloud Run + GPU vs CPU? Return structured JSON.

2. assistant (2026-06-21T13:54:07.524Z)

I'll start by exploring the deploy directory structure to understand what's there.

3. user (2026-06-21T13:54:11.293Z)

```
fatal: Cannot fast-forward to multiple branches.  
---STATUS---  
feat/gcp-ops-agent-and-dryrun-guide
```

4. user (2026-06-21T13:54:11.552Z)

```
total 20  
drwxr-xr-x 1 avidu 197609 0 Jun 21 16:11 .  
drwxr-xr-x 1 avidu 197609 0 Jun 21 19:20 ..  
drwxr-xr-x 1 avidu 197609 0 Jun 21 16:20 cloudrun  
drwxr-xr-x 1 avidu 197609 0 Jun 20 23:04 digitalocean  
drwxr-xr-x 1 avidu 197609 0 Jun 21 19:21 gcp  
===CLOUDRUN===  
total 8  
drwxr-xr-x 1 avidu 197609 0 Jun 21 16:20 .  
drwxr-xr-x 1 avidu 197609 0 Jun 21 16:11 ..  
-rw-r--r-- 1 avidu 197609 3716 Jun 21 16:11 Dockerfile  
-rw-r--r-- 1 avidu 197609 3927 Jun 21 16:20 README.md  
===GCP===  
total 36  
drwxr-xr-x 1 avidu 197609 0 Jun 21 19:21 .  
drwxr-xr-x 1 avidu 197609 0 Jun 21 16:11 ..  
-rw-r--r-- 1 avidu 197609 15690 Jun 21 08:12 README.md  
-rwxr-xr-x 1 avidu 197609 2916 Jun 21 19:21 create_gpu_vm.sh  
-rwxr-xr-x 1 avidu 197609 1274 Jun 21 19:21 ops-agent-startup.sh  
-rwxr-xr-x 1 avidu 197609 3884 Jun 21 19:21 provision.sh  
-rwxr-xr-x 1 avidu 197609 2788 Jun 20 22:51 teardown.sh
```

5. assistant (2026-06-21T13:54:14.013Z)

Let me read the Cloud Run files and the GCP files.

6. user (2026-06-21T13:54:15.001Z)

```
1      # Cloud Run GPU **Job** image for the rally worker (COMPUTE_DECOUPLED_SERVING M6/M7,
ADR-012/D16).
2      #
3      # Runs `python -m backend.worker infer ?` INLINE on an L4 (detect + window + stitch co-
located so
4      # CPU/wall + egress are metered into one job ? D4). Built + pushed + verified by the
owner on the
5      # M7 real run (the $100/?10k budget, D17); CI does NOT build this (a CUDA image is too
heavy) ? the
6      # dispatch shim is proven with a fake client (tests/test_compute_backend.py).
7      #
8      # Two python stacks, deliberately separate (a framework conflict otherwise ? same split
as the
9      # native GPU box, deploy/digitalocean/gpu_setup.sh):
10     # * the MAIN app env (this repo + ffmpeg) drives the pipeline + stitching;
11     # * a conda `wasb` env (py3.8 + torch 1.11.0+cu113) runs the WASB shuttle detector as
a subprocess
12     # (the native runner shells out to $WASB_PYTHON), so the app's torch never co-
installs with it.
13     #
14     # ? WASB-C3: the pretrained badminton weights are academic-data-trained ? provenance NOT
cleared for
15     # commercial sharing. They are NOT baked into this image; stage them at runtime (a
bucket fetch or a
16     # mount) and point $WASB_WEIGHTS at them. Resolve WASB-C3 before any public, non-owner
serving.
17
18     FROM nvidia/cuda:12.4.1-runtime-ubuntu22.04
19
20     ENV DEBIAN_FRONTEND=noninteractive \
21         PYTHONUNBUFFERED=1 \
22         PIP_NO_CACHE_DIR=1 \
23         CONDA_DIR=/opt/conda \
24         WASB_REPO_DIR=/opt/models/WASB-SBDT
25
26     # ffmpeg (stitch/decode) + DejaVu fonts (the watermark filter) + git/curl for the WASB
env build.
27     RUN apt-get update && apt-get install -y --no-install-recommends \
28         ffmpeg fonts-dejavu-core git curl ca-certificates build-essential \
29         python3 python3-pip python3-venv \
30         && rm -rf /var/lib/apt/lists/*
31
32     # --- WASB detector env (mirrors deploy/digitalocean/gpu_setup.sh ? the verified
torch-1.11 stack) ---
33     RUN curl -sSL https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh -o
/tmp/mc.sh \
34         && bash /tmp/mc.sh -b -p "$CONDA_DIR" && rm -f /tmp/mc.sh \
35         && "$CONDA_DIR/bin/conda" tos accept --override-channels --channel
https://repo.anaconda.com/pkgs/main || true \
36         && "$CONDA_DIR/bin/conda" tos accept --override-channels --channel
https://repo.anaconda.com/pkgs/r || true \
37         && "$CONDA_DIR/bin/conda" create -y -q -n wasb python=3.8 \
38         && git clone --depth 1 https://github.com/nttcom/WASB-SBDT.git "$WASB_REPO_DIR" \
39         && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
40             torch==1.11.0+cu113 torchvision==0.12.0+cu113 --extra-index-url
https://download.pytorch.org/whl/cu113 \
41         && ( [ -f "$WASB_REPO_DIR/requirements.txt" ] && "$CONDA_DIR/envs/wasb/bin/python"
-m pip install -q -r "$WASB_REPO_DIR/requirements.txt" || true ) \
42         && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
43             hydra-core omegaconf pandas opencv-python-headless pillow scipy tqdm pyyaml
matplotlib "numpy<1.24"
```

```

44
45 # --- Main application env ---
46 WORKDIR /app
47 COPY pyproject.toml README.md ./
48 COPY backend ./backend
49 COPY training ./training
50 RUN python3 -m pip install --upgrade pip && python3 -m pip install .
51
52 # The native runner finds the WASB env + repo by env (no `conda activate` needed);
weights come in at
53 # runtime (WASB-C3). detector_impl=native = the Linux-native WASB runner; cuda stays the
default.
54 ENV WASB_PYTHON=/opt/conda/envs/wasb/bin/python \
55     WASB_REPO_DIR=/opt/models/WASB-SBDT \
56     WASB_WEIGHTS=/opt/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar \
57     RALLY_DETECTOR_IMPL=native
58
59 # The dispatcher (CloudRunCompute) supplies the per-execution args:
60 # infer --video-uri gs:/// --out-uri gs:/// --done-uri gs:/// --segmenter wasb_hybrid
--set ?
61 ENTRYPOINT ["python3", "-m", "backend.worker"]
62

```

7. user (2026-06-21T13:54:15.324Z)

```

1 # Cloud Run GPU serving ? build / deploy / run (COMPUTE_DECOUPLED_SERVING M6/M7)
2
3 The cloud-serving compute path: a control plane (`CloudRunCompute`,
`backend/compute/cloud_run.py`)
4 dispatches a Cloud Run **Job** execution running this image's worker on an **L4**, then
**bucket-polls**
5 a done-marker for completion (D16). Scale-to-zero (D7) ? no warm pool.
6
7 > **Status:** the dispatch shim + image are **landed + mock-tested** (M6). The **real
build + push +
8 > run on an L4** is the **M7** owner step, within the **$100/?10k** budget (D17) ?
confirm cmd + hourly
9 > cost first, **delete every resource after** ([[gcp-vm-always-delete]]). CI does
**not** build this image.
10
11 ## 0. Prerequisites (owner)
12 - A GCP project with billing on + the Cloud Run, Artifact Registry, and Cloud Build APIs
enabled.
13 - GPU quota for L4 (`GPUS_ALL_REGIONS` ? 1) in the chosen region (asia-south1, else
Singapore ? see
14 `deploy/gcp/README.md`; L4 is often stocked-out, fall back per that runbook).
15 - The WASB weights staged in the bucket (WASB-C3 unresolved ? owner-gated):
`gs://<bucket>/models/wasb_badminton_best.pth.tar`.
16
17 ## 1. Build + push the image (Artifact Registry)
18 ```bash
19 PROJECT=<gcp-project>; REGION=asia-south1; REPO=rally; IMG=rally-worker
20 gcloud artifacts repositories create $REPO --repository-format=docker --location=$REGION
2>/dev/null || true
21 gcloud builds submit --tag $REGION-docker.pkg.dev/$PROJECT/$REPO/$IMG:latest -f
deploy/cloudrun/Dockerfile .
22 ```
23 *(A CUDA + conda image is large; the first build is slow. Budget-watch per D17.)*
24
25 ## 2. Create the Cloud Run **Job** (GPU, scale-to-zero)
26 ```bash
27 gcloud run jobs create rally-worker \
28     --image $REGION-docker.pkg.dev/$PROJECT/$REPO/$IMG:latest \
29     --region $REGION --gpu 1 --gpu-type nvidia-l4 --cpu 4 --memory 16Gi \
30     --max-retries 0 --task-timeout 3600 \

```

```

31      --set-env-vars WASB_WEIGHTS=/staged/wasb_badminton_best.pth.tar
32      # (stage the weights into the container at start, or fetch from gs:// in an init step ?
WASB-C3.)
33      ```
34
35      ## 3. Wire the dispatcher (control plane)
36      The control plane runs with the cloud-serving profile (`DEPLOYMENT_PROFILE=cloud-
serving` ?
37      `compute_target=cloud_run`). `get_compute_backend(config)` then builds `CloudRunCompute`
from env:
38      ```bash
39      export DEPLOYMENT_PROFILE=cloud-serving
40      export CLOUD_RUN_JOB=rally-worker CLOUD_RUN_REGION=$REGION
GOOGLE_CLOUD_PROJECT=$PROJECT
41      # A `worker infer` (or the API job path) with a gs:// video now DISPATCHES to the Job
instead of
42      # running in-process; the container runs the SAME worker INLINE (compute_target forced
local_gpu).
43      python -m backend.worker infer --video-uri gs://<bucket>/videos/clip.mp4 --out-uri
gs://<bucket> --segmenter wasb_hybrid
44      ```
45
46      ## 4. M7 acceptance (the owner runlog)
47      Capture a `DEPLOY_REPORT_cloud-serving_<date>.md` under
48      `docs/COMPUTE_DECOUPLED_SERVING/runlogs/` (mirrors the truly-local template): the exact
commands,
49      rally counts, `gpu_active_seconds`/`wall_seconds`/`bytes_out`, a reel smoke-test
(`ffmpeg -f null -`
50      exit 0 + a 5-frame contact sheet), gotchas, and the teardown audit.
51
52      ## Gotchas
53      - The dispatched container must run inline ? `CloudRunCompute` forces
`compute_target=local_gpu`
54      + `indexing.detector_impl=native` in the per-execution args so the Job never re-
dispatches itself.
55      - Completion is a bucket marker (`<out_uri>/_dispatch/<token>.done`), not the Cloud
Run API status
56      ? so it reuses the storage layer + `execution_policy.poll_until` and survives a
control-plane restart.
57      - A failed Job currently writes no marker ? the dispatcher's bounded poll times
out
58      (`ExecutionPolicy.max_wait_sec`) and `_dispatch_remote` returns a clean rc 4 (not
a traceback).
59      A failure-marker (carrying the real non-zero rc, so the dispatcher distinguishes
failed from
60      hung/cold-start-slow) is a follow-up.
61      - libx264 encode (parity, D16) + NVDEC decode-only; the image keeps ffmpeg's default
libx264.
62

```

8. assistant (2026-06-21T13:54:18.225Z)

Now let me look at the GCP scripts to check for any Cloud Run bits there, and check for build/push scripts and CI.

9. user (2026-06-21T13:54:18.887Z)

```

1      #!/usr/bin/env bash
2      # Provision the DECOUPLED_COMPUTE GCP storage + IAM (asia-south1 / Mumbai). Idempotent:
safe to
3      # re-run; already-existing resources are left as-is. The GPU VM is NOT created here (it
is gated on
4      # the GPUS_ALL_REGIONS quota ? see deploy/gcp/README.md §2-3); create it with the $3
command once
5      # the quota is granted. This documents EXACTLY what was run on 2026-06-20 (D0->GCP

```

```

pivot, ADR-010).
6      #
7      # Prereq: `gcloud auth login`; Service Usage API + a linked billing account enabled in
the console
8      # (README §1). Run: bash deploy/gcp/provision.sh
9      set -uo pipefail
10
11     PROJECT="${RALLY_GCP_PROJECT:-gen-lang-client-0306026826}"
12     REGION="${RALLY_GCP_REGION:-asia-south1}"
13     BUCKET="${RALLY_GCS_BUCKET:-khelsutra-rally-corpus}"
14     BILLING_ACCT="${RALLY_BILLING_ACCT:-01420D-419EE0-53D83C}"
15     SA_NAME="rally-worker"
16     SA_EMAIL="${SA_NAME}@${PROJECT}.iam.gserviceaccount.com"
17     SA_KEY="${RALLY_SA_KEY:-$HOME/.gcp/rally-worker-sa.json}"
18     export CLOUDSDK_CORE_DISABLE_PROMPTS=1
19
20     echo "== project=$PROJECT region=$REGION bucket=gs://$BUCKET =="
21
22     echo "== enable APIs =="
23     # logging + monitoring are REQUIRED for the Ops Agent ? without them the agent installs
but its
24     # metric/log exports get PermissionDenied/SERVICE_DISABLED (the empty Observability
graphs cause).
25     gcloud services enable compute.googleapis.com cloudbilling.googleapis.com \
26         billingbudgets.googleapis.com iam.googleapis.com cloudresourcemanager.googleapis.com \
27         logging.googleapis.com monitoring.googleapis.com \
28         --project "$PROJECT"
29
30     echo "== GCS bucket (asia-south1, uniform bucket-level access) =="
31     gcloud storage buckets create "gs://$BUCKET" --location="$REGION" \
32         --uniform-bucket-level-access --project "$PROJECT" || echo " (bucket exists ? ok)"
33
34     echo "== service account + Storage Object Admin + key =="
35     gcloud iam service-accounts create "$SA_NAME" --display-name="Rally worker (storage)" \
36         --project "$PROJECT" || echo " (SA exists ? ok)"
37     gcloud projects add-iam-policy-binding "$PROJECT" \
38         --member="serviceAccount:$SA_EMAIL" --role="roles/storage.objectAdmin"
--condition=None \
39         --format="value(etag)" >/dev/null
40     # Ops Agent (GPU/NVML metrics + logs ? Cloud Monitoring/Logging) needs these on the VM's
SA, else
41     # its exports are PermissionDenied. Granted here so EVERY VM (created via
create_gpu_vm.sh) reports.
42     echo "== Ops Agent SA roles (monitoring.metricWriter + logging.logWriter) =="
43     for ROLE in roles/monitoring.metricWriter roles/logging.logWriter; do
44         gcloud projects add-iam-policy-binding "$PROJECT" \
45             --member="serviceAccount:$SA_EMAIL" --role="$ROLE" --condition=None \
46             --format="value(etag)" >/dev/null
47     done
48     if [ ! -f "$SA_KEY" ]; then
49         mkdir -p "$(dirname "$SA_KEY")"
50         gcloud iam service-accounts keys create "$SA_KEY" --iam-account="$SA_EMAIL"
51         echo " key -> $SA_KEY (NEVER commit this; export
GOOGLE_APPLICATION_CREDENTIALS=$SA_KEY)"
52     else
53         echo " key already at $SA_KEY ? leaving it (delete to rotate)"
54     fi
55
56     echo "== \$100 budget alert (50/90/100%) =="
57     gcloud billing budgets create --billing-account="$BILLING_ACCT" \
58         --display-name="rally-100usd-guard" --budget-amount=100 \
59         --filter-projects="projects/$PROJECT" \
60         --threshold-rule=percent=0.5 --threshold-rule=percent=0.9 --threshold-rule=percent=1.0
\
61     || echo " (budget may already exist ? check: gcloud billing budgets list --billing-

```

```

account=$BILLING_ACCT)"
62
63     echo "== GPU quota (the VM gate) =="
64     echo "    regional $REGION:"
65     gcloud compute regions describe "$REGION" --project "$PROJECT" \
66         --flatten="quotas[]" --format="table(quotas.metric,quotas.limit)" | grep -E
"NVIDIA_(T4|L4)_GPUS " || true
67     echo "    global:"
68     gcloud compute project-info describe --project "$PROJECT" \
69         --flatten="quotas[]" --format="table(quotas.metric,quotas.limit)" | grep
GPUS_ALL_REGIONS || true
70     echo ""
71     echo "If GPUS_ALL_REGIONS == 0, request an increase (README §2), then create the VM
(README §3)."
```

10. user (2026-06-21T13:54:19.257Z)

```

1     #!/usr/bin/env bash
2     # Create a rally GPU VM that ALWAYS installs the Ops Agent (Observability). This is THE
one way to
3     # spin up a rally GPU box ? it bakes the ops-agent startup-script into the VM metadata,
attaches the
4     # rally-worker service account (+ cloud-platform scope so it can read/write the bucket +
report
5     # metrics), and uses a driver-ready Deep-Learning-VM image (nvidia-smi works
immediately).
6     #
7     # It sweeps zones for GPU capacity ? asia-south1 (Mumbai, co-located with the bucket)
first, then
8     # asia-southeast1 (Singapore), which L4 frequently falls back to when Mumbai is stocked-
out.
9     #
10    #   bash deploy/gcp/create_gpu_vm.sh                                # L4 (default), name
rally-gpu
11    #   RALLY_VM_NAME=rally-m7 RALLY_GPU=t4 bash deploy/gcp/create_gpu_vm.sh
12    #
13    # Prereq (once per project): bash deploy/gcp/provision.sh   (enables logging/monitoring
+ grants the
14    # SA the Ops-Agent writer roles) AND the GPUS_ALL_REGIONS quota >= 1 (README §2).
15    #
16    # ? DELETE when done (a running GPU VM is the only real cost):
17    #   RALLY_GCP_ZONE=<zone-it-printed> bash deploy/gcp/teardown.sh delete
18    set -uo pipefail
19
20    PROJECT="${RALLY_GCP_PROJECT:-gen-lang-client-0306026826}"
21    NAME="${RALLY_VM_NAME:-rally-gpu}"
22    SA="rally-worker@${PROJECT}.iam.gserviceaccount.com"
23    GPU="${RALLY_GPU:-l4}"      # l4 | t4
24    HERE="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
25    STARTUP="$HERE/ops-agent-startup.sh"
26    export CLOUDSDK_CORE_DISABLE_PROMPTS=1
27
28    ZONES="${RALLY_GCP_ZONES:-asia-south1-a asia-south1-b asia-south1-c asia-southeast1-a
asia-southeast1-b asia-southeast1-c}"
29
30    if [ "$GPU" = "l4" ]; then
31        MACHINE="g2-standard-4"; ACCEL=(                # L4 is integral to the g2 machine family
32    else
33        MACHINE="n1-standard-8"; ACCEL=(--accelerator=type=nvidia-tesla-t4,count=1)
34    fi
35
36    ERR="$(mktemp)"
37    for Z in $ZONES; do
38        echo "== trying $NAME ($GPU) in $Z =="
```

```

39     if gcloud compute instances create "$NAME" --project="$PROJECT" --zone="$Z" \
40         --machine-type="$MACHINE" "${ACCEL[@]}" \
41         --maintenance-policy=TERMINATE --provisioning-model=STANDARD \
42         --image-family=common-cu129-ubuntu-2204-nvidia-580 \
43         --image-project=deeplearning-platform-release \
44         --boot-disk-size=120GB --boot-disk-type=pd-balanced \
45         --service-account="$SA" --scopes=cloud-platform \
46         --metadata-from-file=startup-script="$STARTUP" 2>"$ERR"; then
47     echo "? CREATED $NAME in $Z ? the Ops Agent installs on boot (Observability ~2-3 min
later)."
48     echo "    SSH:    gcloud compute ssh $NAME --zone=$Z"
49     echo "    DELETE: RALLY_GCP_ZONE=$Z bash deploy/gcp/teardown.sh delete"
50     rm -f "$ERR"; exit 0
51 fi
52 if grep -qiE "STOCKOUT|does not have enough" "$ERR"; then
53     echo "    (no $GPU capacity in $Z ? trying the next zone)"; continue
54 fi
55 echo "    create failed:"; tail -4 "$ERR"; rm -f "$ERR"; exit 1
56 done
57 echo "? No $GPU capacity in any swept zone ($ZONES). Try again later or RALLY_GPU=t4."
58 rm -f "$ERR"; exit 1
59

```

11. assistant (2026-06-21T13:54:21.549Z)

Now let me check the GCP README for Cloud Run sections, and search for any Cloud Run references in CI and across the repo.

12. user (2026-06-21T13:54:23.561Z)

No matches found

13. user (2026-06-21T13:54:24.877Z)

Found 7 files
docs\COMPUTE_DECOUPLED_SERVING\README.md
tests\test_compute_backend.py
deploy\cloudrun\README.md
backend\compute\cloud_run.py
deploy\cloudrun\Dockerfile
backend\compute__init__.py
docs\archives\past_projects\DECOUPLED_COMPUTE\05-COST-ATTRIBUTION.md

14. assistant (2026-06-21T13:54:29.236Z)

Let me check the CI workflows for any build of this image, and verify the worker entrypoint and the GPU/CPU handling in the compute backend.

15. user (2026-06-21T13:54:30.348Z)

```

total 8
drwxr-xr-x 1 avidu 197609    0 Jun 17 19:44 .
drwxr-xr-x 1 avidu 197609    0 Jun 10 11:31 ..
-rw-r--r-- 1 avidu 197609 5597 Jun 17 19:44 ci.yml

```

16. user (2026-06-21T13:54:31.769Z)

```

.github\workflows\ci.yml:80:      - name: Install CPU-only torch (no CUDA wheels on a CPU
runner)
.github\workflows\ci.yml:81:      # torch/torchvision are NOT in pyproject deps ? their
CPU/CUDA wheels

```

17. assistant (2026-06-21T13:54:36.372Z)

CI does not build the Docker image. Let me verify the worker entrypoint and the GPU/CPU device handling in the compute backend to confirm the "worker vs server" and "GPU vs CPU" story.

18. user (2026-06-21T13:54:37.320Z)

```
1      """`CloudRunCompute` ? dispatch one inference job to a Cloud Run GPU **Job** and
bucket-poll
2      for its completion (COMPUTE_DECOUPLED_SERVING M6, decisions D16 + D7).
3
4      Flow (the cloud-serving compute path):
5      1. **Dispatch** a Cloud Run Job execution whose container runs ``python -m
backend.worker infer
6      --video-uri <gs> --out-uri <gs> --done-uri <gs> ?`` (the SAME worker, on the L4
GPU).
7      2. **Bucket-poll** ``done_uri`` via ``execution_policy.poll_until`` + the storage
layer (D16 ? the
8      completion signal is a tiny marker object the worker writes, NOT the Cloud Run API
status, so
9      it is storage-backed and reuses the existing park/resume primitives). Scale-to-zero
(D7): the
10     cold-start is absorbed by the async poll.
11     3. **Read** the marker (it carries ``video_id`` + the worker's returncode) ? fetch
12     ``outputs/<video_id>/report.json`` ? ``ComputeResult``.
13
14     The dispatched container is forced to run **INLINE** (``compute_target=local_gpu`` +
native detector)
15     so it never recursively re-dispatches ? the dispatcher decides cloud, the container just
computes.
16
17     The Cloud Run trigger is an **injected client** (``CloudRunJobClient``) so the whole
shim is unit-tested
18     with a fake (no GCP). The real ``GoogleCloudRunJobClient`` lazy-imports ``google-cloud-
run`` and is
19     owner-verified on the M7 real run.
20     """
21
22     from __future__ import annotations
23
24     import json
25     import logging
26     import os
27     import tempfile
28     import uuid
29     from abc import ABC, abstractmethod
30     from typing import Any, List, Optional
31
32     from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
33     from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy,
poll_until
34     from backend.storage import fetch, parse_uri
35     from backend.storage import get_storage
36
37     LOG = logging.getLogger("compute.cloud_run")
38
39     # The container worker must run INLINE on the GPU ? never re-enter the cloud-dispatch
branch.
40     # (compute_target=local_gpu disarms the cmd_infer dispatch guard; native = the L4 WASB
runner.)
41     _DEFAULT_CONTAINER_OVERRIDES: tuple[str, ...] = (
42         "deployment.compute_target=local_gpu",
43         "indexing.detector_impl=native",
44     )
45
46     class CloudRunJobClient(ABC):
```



```

48         """Triggers a Cloud Run **Job** execution with per-execution container arg
overrides.
49
50         Abstracted so ``CloudRunCompute`` is testable with a fake (no GCP). A real
implementation
51         overrides the job's container ``args`` for this execution and starts it."""
52
53         @abstractmethod
54         def run_job(self, job_name: str, argv: List[str], *, region: str,
55                     project: Optional[str] = None) -> str:
56             """Start a Cloud Run Job execution running ``python -m backend.worker <argv>``;
return an
57             execution id/name (for logs). Does NOT block on completion (the caller bucket-
polls)."""
58
59
60         class GoogleCloudRunJobClient(CloudRunJobClient):
61             """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real
run; CI uses
62             a fake, so this code path is never exercised in tests (the SDK isn't a test
dependency)."""
63
64             def run_job(self, job_name: str, argv: List[str], *, region: str,
65                         project: Optional[str] = None) -> str:
66                 try:
67                     from google.cloud import run_v2 # type: ignore[attr-defined]
68                 except Exception as exc: # pragma: no cover - real SDK not present in CI
69                     raise RuntimeError(
70                         "google-cloud-run is required for CloudRunCompute's real client; install
it on the "
71                         "control plane (it is intentionally NOT a CI/test dependency).")
72                 ) from exc
73                 client = run_v2.JobsClient() # pragma: no cover - exercised only on the real M7
run
74                 name = client.job_path(project, region, job_name)
75                 overrides = run_v2.RunJobRequest.Overrides(
76                     container_overrides=[run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)]
77                 )
78                 op = client.run_job(request=run_v2.RunJobRequest(name=name,
overrides=overrides))
79                 return getattr(op, "metadata", None) and getattr(op.metadata, "name", "") or
"execution"
80
81
82         class CloudRunCompute(ComputeBackend):
83             """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""
84
85             def __init__(self, *, run_client: CloudRunJobClient, job_name: str, region: str,
86                         project: Optional[str] = None, storage_config: Optional[dict] = None,
87                         policy: ExecutionPolicy = DEFAULT_POLICY,
88                         token: Optional[str] = None,
89                         container_overrides: Optional[tuple[str, ...]] = None) -> None:
90                 self._client = run_client
91                 self._job_name = job_name
92                 self._region = region
93                 self._project = project
94                 self._storage_config = storage_config or {}
95                 self._policy = policy
96                 # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
collide.
97                 # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic
(tests).
98                 self._token = token
99                 self._container_overrides = (

```

```

100         _DEFAULT_CONTAINER_OVERRIDES if container_overrides is None else
tuple(container_overrides)
101     )
102
103     def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
104         out_root = spec.out_uri.rstrip("/")
105         token = self._token or uuid.uuid4().hex
106         done_uri = f"{out_root}/_dispatch/{token}.done"
107         argv: List[str] = ["infer", "--video-uri", spec.video_uri, "--out-uri",
spec.out_uri,
108                             "--done-uri", done_uri]
109         if spec.segmenter:
110             argv += ["--segmenter", spec.segmenter]
111         for kv in (*spec.config_overrides, *self._container_overrides):
112             argv += ["--set", kv]
113
114         execution = self._client.run_job(self._job_name, argv, region=self._region,
115                                         project=self._project)
116         LOG.info("cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
117                 self._job_name, execution, done_uri)
118
119         marker = poll_until(lambda: self._read_marker(done_uri), self._policy,
120                             label="cloud-run-infer", logger=LOG)
121         video_id = str(marker.get("video_id", ""))
122         # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
123         # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
124         rc = int(marker.get("returncode", 1))
125         if not video_id:
126             LOG.warning("cloud-run: job=%s completion marker present but
empty/unreadable (no "
127                         "video_id) ? treating as failure", self._job_name)
128             rc = rc or 1
129         outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
130         report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
131         LOG.info("cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id,
rc)
132         return ComputeResult(returncode=rc, outputs_uri=outputs_uri, video_id=video_id,
report=report)
133
134     # --- storage-backed completion signal (bucket-poll) ---
135     def _read_marker(self, done_uri: str) -> Optional[dict]:
136         """The done-marker dict if the worker has written it, else None ? poll_until
keeps waiting."""
137         ref = parse_uri(done_uri)
138         if not get_storage(ref.backend, config=self._storage_config).exists(ref):
139             return None
140         return self._read_json(done_uri) or {} # empty dict still terminates the poll
(present == done)
141
142     def _read_json(self, uri: str) -> dict:
143         # Download to a real local dst FIRST: the cloud backends (gcs/s3) REQUIRE a dst
? a None dst
144         # raises ValueError ? unlike local's identity passthrough. Without an explicit
dst this read
145         # would silently return {} for every gs://|s3:// object (the only backends this
shim targets).
146         try:
147             ref = parse_uri(uri)
148             dst = os.path.join(tempfile.mkdtemp(prefix="cr-read-"), ref.name or
"obj.json")
149             local = fetch(uri, dst, config=self._storage_config)
150             with open(local, "r", encoding="utf-8") as f:
151                 data: Any = json.load(f)

```

```

152         return data if isinstance(data, dict) else {}
153     except Exception:
154         LOG.warning("cloud-run: could not read %s", uri, exc_info=True)
155         return {}
156

```

19. user (2026-06-21T13:54:39.079Z)

```

__init__.py
__main__.py
__pycache__
---

```

head: cannot open 'C:/Users/avidu/Projects/badminton-highlight-indexer/backend/worker.py' for reading: No such file or directory

```

__init__.py
__main__.py
__pycache__

```

20. assistant (2026-06-21T13:54:42.189Z)

The worker is a package. Let me check the worker `\_\_main\_\_` to confirm the entrypoint args and the `--done-uri` marker writing, plus check how device/GPU context is handled (the M4 DeviceContext from the branch context).

21. user (2026-06-21T13:54:43.777Z)

```

517 C:/Users/avidu/Projects/badminton-highlight-indexer/backend/worker/__main__.py
7 C:/Users/avidu/Projects/badminton-highlight-indexer/backend/worker/__init__.py
524 total

```

22. user (2026-06-21T13:54:45.163Z)

```

backend\worker\__main__.py:3: python -m backend.worker infer --video-uri <uri> --out-uri
<prefix> \
backend\worker\__main__.py:40:from backend.utils.hashing import compute_video_id
backend\worker\__main__.py:86:def _write_report_json(video_id: str, segments: List[dict],
status: str, path: str) -> None:
backend\worker\__main__.py:89:         "video_id": video_id,
backend\worker\__main__.py:104:     torch, and pre-M5 the worker's infer/train path was torch-
free (detection is delegated to a
backend\worker\__main__.py:117:     """M6: hand ONE infer job to a remote ComputeBackend (Cloud
Run) and return its rc once the
backend\worker\__main__.py:119:     container runs this SAME cmd_infer INLINE (compute_target
forced local_gpu) ? never re-dispatch.
backend\worker\__main__.py:131:         result = compute.run_infer(spec)
backend\worker\__main__.py:135:     logger.info("[worker] remote compute done: video_id=%s rc=%d
outputs=%s",
backend\worker\__main__.py:136:         result.video_id, result.returncode,
result.outputs_uri)
backend\worker\__main__.py:137:     return result.returncode
backend\worker\__main__.py:140:def _write_done_marker(done_uri: str, video_id: str, returncode:
int, segment_count: int,
backend\worker\__main__.py:142:     """M6: write a tiny JSON completion marker to ``done_uri``
(via the storage layer) for a remote
backend\worker\__main__.py:145:         marker = {"video_id": video_id, "returncode": returncode,
backend\worker\__main__.py:150:         storage.put(local, done_uri, config=storage_cfg)
backend\worker\__main__.py:151:         logger.info("[worker] wrote completion marker -> %s",
done_uri)
backend\worker\__main__.py:153:         logger.warning("[worker] could not write done-marker %s:
%s", done_uri, e)
backend\worker\__main__.py:156:def cmd_infer(args: argparse.Namespace) -> int:
backend\worker\__main__.py:162:     # M6: a cloud compute_target DISPATCHES to a Cloud Run Job
instead of running in-process.
backend\worker\__main__.py:164:     # unchanged inline path below. The dispatched container runs

```

```

THIS cmd_infer inline (the
backend\worker\_main_.py:165:    # dispatcher forces compute_target=local_gpu), so it never
recursively re-dispatches.
backend\worker\_main_.py:181:    video_id = compute_video_id(local_video)
backend\worker\_main_.py:187:    db.add_video(video_id, local_video, "failed" if failures else
"processed",
backend\worker\_main_.py:206:        result = segmenter_cls(db,
config).process_video(video_id, local_video, max_frames=args.max_frames)
backend\worker\_main_.py:217:        detector_impl =
(os.environ.get("RALLY_DETECTOR_IMPL")
backend\worker\_main_.py:218:            or getattr(config.indexing,
"detector_impl", None) or "auto")
backend\worker\_main_.py:220:            "infer produced 0 segments for video_id=%s
(segmenter=%s, detector_impl=%s). "
backend\worker\_main_.py:225:            video_id, segmenter_actual, detector_impl,
backend\worker\_main_.py:230:            db=db, video_id=video_id, video_path=local_video,
config=config,
backend\worker\_main_.py:236:    # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent
on video_id.
backend\worker\_main_.py:237:    out_local = os.path.join(scratch, "outputs", video_id)
backend\worker\_main_.py:240:    _write_report_json(video_id, segments, status,
os.path.join(out_local, "report.json"))
backend\worker\_main_.py:245:    uri = f"{out_prefix}/outputs/{video_id}/{fn}"
backend\worker\_main_.py:249:    print(f"[worker] infer done: {len(segments)} segment(s);
pushed {len(pushed)} artifact(s):")
backend\worker\_main_.py:253:    # M6: a remote dispatcher passes --done-uri; write a
completion marker so its bucket-poll
backend\worker\_main_.py:254:    # terminates (carries video_id ? the dispatcher's output path
+ the worker rc). DEFAULT-OFF:
backend\worker\_main_.py:255:    # no --done-uri ? no marker (unchanged local run). Written on
SUCCESS only ? a failed container
backend\worker\_main_.py:257:    if getattr(args, "done_uri", None):
backend\worker\_main_.py:258:        _write_done_marker(args.done_uri, video_id, 0,
len(segments), status, storage_cfg, scratch)
backend\worker\_main_.py:273:    inference report (it flags the fallback) but corrupting for
REAL training data. So here we

```

[Showing results with pagination = limit: 40]

23. assistant (2026-06-21T13:54:47.961Z)

This confirms the full picture. Let me check the device/GPU handling story (DeviceContext from the M4 branch) to ground the GPU vs CPU description, and confirm the native WASB runner's device handling.

24. user (2026-06-21T13:54:49.939Z)

Found 3 files

```

backend\pipeline\segmenters\trajectory_hybrid.py
backend\pipeline\segmenters\fusion_hybrid.py
backend\pipeline\segmenters\wasb_hybrid.py

```

25. user (2026-06-21T13:54:51.530Z)

```

<tool_use_error>Path does not exist: C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\device_context.py. Note: your current working directory is
C:\Users\avidu\Projects\badminton-highlight-indexer.</tool_use_error>

```

26. assistant (2026-06-21T13:54:54.583Z)

I have enough to ground the device/GPU story from the Dockerfile itself plus the worker context. Let me quickly confirm where DeviceContext lives and the native runner's CPU-fallback behavior, since the current branch is about that.

27. user (2026-06-21T13:54:56.062Z)

Found 4 files
backend\config\models.py
backend\config\startup.py
backend\pipeline\detectors\native_wasb_runner.py
backend\pipeline\detectors\device.py

28. assistant (2026-06-21T13:55:00.316Z)

I have a complete picture now. The Dockerfile hardcodes the WASB env's torch as `cu113` (GPU CUDA build) and the base image is a CUDA runtime, so the image is GPU-oriented; the CPU-fallback (device="auto") is a code-level capability in the native runner, not something the Cloud Run image is built to exploit (Cloud Run Job is created with `--gpu 1 --gpu-type nvidia-l4`). I now have everything needed to answer.

29. user (2026-06-21T13:55:51.918Z)

Structured output provided successfully

30. assistant (2026-06-21T13:56:02.815Z)

I've completed the analysis of the Cloud Run containerization and deploy setup. Summary of what I found:

****Dockerfile**** (`deploy/cloudrun/Dockerfile`): Single GPU worker image on `nvidia/cuda:12.4.1-runtime-ubuntu22.04`. Two separate python stacks ? the main app env (system python3, `pip install .`, no torch) and a conda `wasb` env (py3.8 + torch 1.11.0+cu113) cloned from nttcom/WASB-SBDT and run as a subprocess via `\$WASB\_PYTHON`. Entrypoint is the ****worker**** (`python3 -m backend.worker`), not the server; the dispatcher injects `infer --video-uri ? --done-uri ?` args per execution. WASB weights are deliberately NOT baked in (WASB-C3 provenance).

****Buildable vs scaffold****: It's a scaffold/owner-built-once. CI (`github/workflows/ci.yml`) does NOT build it (CPU-only mocked tests; header says "CI does NOT build this"). No build/push/deploy script exists ? only copy-paste `gcloud builds submit` / `gcloud run jobs create` commands in `deploy/cloudrun/README.md`. The control-plane dispatch shim (`backend/compute/cloud\_run.py`) is landed and mock-tested, but the real `GoogleCloudRunJobClient` and the actual L4 build/run are owner-verified only (the M7 step).

****GPU vs CPU****: GPU-first and Cloud-Run-****Job****-only ? deployed with `--gpu 1 --gpu-type nvidia-l4`, scale-to-zero. No CPU variant of the image; the M4 `DeviceContext`/CPU-fallback (`backend/pipeline/detectors/device.py`) is a code-level capability, not a buildable CPU deploy target.

****deploy/gcp****: Contains zero Cloud Run wiring (grep-confirmed) ? those scripts only provision a GPU VM + bucket + IAM + budget.

Full details (file paths, deploy commands, GPU story, and seven gaps) returned via the StructuredOutput tool.